

MANUAL BOOK TEKNIS

TODO LIST MANAGEMENT API

Panduan Instalasi, Konfigurasi, dan Penggunaan REST API

Dibangun dengan Node.js · Express.js · MongoDB (Mongoose)

Jenis Dokumen	Manual Book Teknis (Panduan Penggunaan)
Versi Dokumen	1.0
Proyek	Todo List Management API
Arsitektur	Controller - Service - Model

Daftar Isi

1. Pendahuluan	3
2. Persyaratan Sistem	4
3. Instalasi & Konfigurasi	5
4. Menjalankan Aplikasi	6
5. Struktur Proyek	7
6. Autentikasi & Otorisasi	8
7. Panduan Penggunaan Endpoint API	9
8. Format Response & Error Handling	12
9. Pengujian Manual dengan Postman	13
10. Dokumentasi API Interaktif (Swagger)	14
11. Troubleshooting (Masalah Umum)	15
12. Penutup	16

1. Pendahuluan

1.1 Deskripsi Proyek

Todo List Management API adalah REST API production-ready untuk mengelola daftar tugas (todo), kategori, pengguna, dan log aktivitas. API ini dibangun menggunakan arsitektur Controller-Service-Model yang modular, sehingga mudah dipelihara dan dikembangkan lebih lanjut.

1.2 Fitur Utama

- Autentikasi pengguna menggunakan JWT (JSON Web Token) dengan password ter-hash (bcrypt).
- Otorisasi berbasis role (Role-Based Access Control) — admin vs user.
- Proteksi endpoint machine-to-machine menggunakan API Key (tanpa JWT).
- CRUD lengkap untuk Todo dan Category, masing-masing dimiliki oleh user yang login.
- Pagination, sorting, filtering, dan search pada endpoint list (Todo, Category, User, Activity Log).
- Soft delete — data yang dihapus tidak benar-benar hilang, hanya ditandai archived.
- Pencatatan otomatis Activity Log setiap ada aksi create/update/delete pada Todo.
- Validasi input menggunakan express-validator pada seluruh endpoint.
- Global error handler dengan format response yang konsisten dan HTTP status code presisi.
- Dokumentasi API interaktif menggunakan Swagger/OpenAPI.
- Automated testing menggunakan Jest dan Supertest.
- Database seeder untuk data awal (admin, user, kategori, dan todo contoh).

1.3 Teknologi yang Digunakan

Kategori	Teknologi
Runtime	Node.js (v18+)
Framework	Express.js
Database	MongoDB + Mongoose ODM
Autentikasi	jsonwebtoken, bcryptjs
Validasi	express-validator
Dokumentasi	swagger-jsdoc, swagger-ui-express
Keamanan	helmet, cors, express-rate-limit, express-mongo-sanitize, xss-clean, hpp
Pengujian	Jest, Supertest, mongodb-memory-server

2. Persyaratan Sistem

- Node.js versi 18 atau lebih baru.
- MongoDB (lokal atau layanan cloud seperti MongoDB Atlas).
- npm (terpasang bersama Node.js).
- Postman (opsional, untuk pengujian manual endpoint).
- Git (untuk version control dan upload ke GitHub).

3. Instalasi & Konfigurasi

3.1 Ekstrak / Clone Proyek

Ekstrak file proyek (atau clone dari repository GitHub), lalu masuk ke folder proyek:

```
cd todo-list-api
```

3.2 Instalasi Dependencies

Jalankan perintah berikut untuk menginstal seluruh library yang dibutuhkan:

```
npm install
```

3.3 Konfigurasi Environment Variable

Salin file .env.example menjadi .env, lalu sesuaikan isinya:

```
cp .env.example .env
```

Berikut penjelasan variabel environment yang tersedia:

Variabel	Keterangan	Contoh
NODE_ENV	Mode aplikasi	development / production / test
PORT	Port server	5000
API_PREFIX	Prefix seluruh endpoint API	/api/v1
MONGO_URI	Connection string MongoDB	mongodb://127.0.0.1:27017/todo_list_db
MONGO_URI_TEST	Connection string MongoDB untuk testing	mongodb://127.0.0.1:27017/todo_list_db_test
JWT_SECRET	Kunci rahasia untuk menandatangani JWT	string acak yang panjang
JWT_EXPIRES_IN	Masa berlaku token JWT	1d
API_KEY	Kunci statis untuk endpoint machine-to-machine	string acak yang panjang
RATE_LIMIT_WINDOW_MS	Jendela waktu rate limiting (ms)	900000
RATE_LIMIT_MAX	Maksimal request per IP per jendela waktu	100
BCRYPT_SALT_ROUNDS	Cost factor hashing bcrypt	10

4. Menjalankan Aplikasi

4.1 Mode Development

Menjalankan server dengan auto-reload (nodemon):

```
npm run dev
```

4.2 Mode Production

```
npm start
```

Setelah server berjalan, API dapat diakses di <http://localhost:5000/api/v1> dan dokumentasi Swagger tersedia di <http://localhost:5000/api-docs>.

4.3 Menjalankan Seeder (Data Awal)

Untuk mengisi database dengan data awal (1 admin, 1 user, 3 kategori, dan 3 todo per kategori):

```
npm run seed
```

Kredensial akun hasil seeding:

Role	Email	Password
Admin	admin@example.com	Admin@123
User	user@example.com	User@123

Untuk menghapus data hasil seeding:

```
node utils/seed.js --clear
```

4.4 Menjalankan Automated Test

```
npm test
```

Pengujian menggunakan mongodb-memory-server (tidak memerlukan MongoDB eksternal). Jika binary tidak dapat diunduh (mis. jaringan terbatas), sistem otomatis fallback menggunakan MONGO_URI_TEST.

5. Struktur Proyek

```
todo-list-api/  
├── config/           # Koneksi DB, environment, konfigurasi Swagger  
├── controllers/      # Request/response handler  
├── services/         # Logika bisnis & query Mongoose  
├── models/           # Skema Mongoose (User, Category, Todo, ActivityLog)  
├── middlewares/      # Auth, RBAC, API Key, logger, notFound, validator, error handler  
├── routes/           # Definisi endpoint + dokumentasi Swagger  
├── validations/      # Rule validasi express-validator  
├── utils/            # AppError, catchAsync, apiFeatures, seeder  
├── tests/            # Unit/integration test (Jest + Supertest)  
├── postman/          # Postman collection & environment  
├── app.js            # Konfigurasi Express app  
├── server.js         # Entry point aplikasi  
└── README.md
```

6. Autentikasi & Otorisasi

6.1 Alur Autentikasi

1. Daftar akun melalui POST /api/v1/auth/register.
2. Login melalui POST /api/v1/auth/login untuk mendapatkan JWT access token.
3. Sertakan token pada header Authorization di setiap request yang membutuhkan autentikasi:

```
Authorization: Bearer <token>
```

6.2 Role-Based Access Control (RBAC)

Setiap user memiliki role "user" (default) atau "admin". Endpoint di bawah /api/v1/users/* hanya bisa diakses oleh role admin, sebagai contoh penerapan middleware protect + restrictTo("admin").

6.3 API Key (Machine-to-Machine)

Endpoint di bawah /api/v1/external/* tidak memerlukan JWT, melainkan API Key statis yang dikirim melalui header berikut:

```
x-api-key: <API_KEY>
```


7. Panduan Penggunaan Endpoint API

7.1 Auth

Method	Endpoint	Auth	Deskripsi
POST	/api/v1/auth/register	Public	Registrasi akun baru
POST	/api/v1/auth/login	Public	Login, menghasilkan JWT
GET	/api/v1/auth/me	JWT	Profil user yang sedang login

Contoh Request — Register

```
{
  "name": "Jane Doe",
  "email": "jane@example.com",
  "password": "password123"
}
```

Contoh Response — 201 Created

```
{
  "success": true,
  "message": "User registered successfully.",
  "data": {
    "user": {
      "_id": "...",
      "name": "Jane Doe",
      "email": "jane@example.com",
      "role": "user"
    },
    "token": "<jwt-token>"
  }
}
```

7.2 Users (Admin Only)

Method	Endpoint	Auth	Deskripsi
GET	/api/v1/users	JWT (Admin)	Daftar seluruh user (pagination & search)
GET	/api/v1/users/:id	JWT (Admin)	Detail user berdasarkan ID
DELETE	/api/v1/users/:id	JWT (Admin)	Soft-delete (arsipkan) user

7.3 Categories

Method	Endpoint	Auth	Deskripsi
GET	/api/v1/categories	JWT	Daftar kategori milik user (pagination & search)
POST	/api/v1/categories	JWT	Membuat kategori baru

Method	Endpoint	Auth	Deskripsi
GET	/api/v1/categories/:id	JWT	Detail kategori
PUT	/api/v1/categories/:id	JWT	Memperbarui kategori
DELETE	/api/v1/categories/:id	JWT	Soft-delete (arsipkan) kategori

7.4 Todos

Method	Endpoint	Auth	Deskripsi
GET	/api/v1/todos	JWT	Daftar todo (pagination, sorting, filter, search)
POST	/api/v1/todos	JWT	Membuat todo baru
GET	/api/v1/todos/:id	JWT	Detail todo
PUT	/api/v1/todos/:id	JWT	Memperbarui todo
DELETE	/api/v1/todos/:id	JWT	Soft-delete (arsipkan) todo

Contoh Query — Pagination, Sorting, Filtering, Search

```
GET  
/api/v1/todos?page=1&limit=10&field=createdAt&order=desc&status=pending&category=<categoryId>&search=laporan
```

Penjelasan Parameter Query

Parameter	Keterangan
page	Halaman yang ditampilkan (default: 1)
limit	Jumlah data per halaman (default: 10, maksimal 100)
field	Nama field untuk sorting, contoh: createdAt
order	Arah sorting: asc atau desc
category	Filter berdasarkan ObjectId kategori
status	Filter berdasarkan status: pending, in-progress, completed
search	Kata kunci pencarian pada judul/deskripsi todo

7.5 Activity Logs

Method	Endpoint	Auth	Deskripsi
GET	/api/v1/activity-logs	JWT	Log aktivitas (admin melihat semua, user hanya miliknya)

Setiap aksi create, update, atau delete pada Todo otomatis dicatat pada koleksi ActivityLog, mencakup jenis aksi, entitas, waktu, dan pelaku aksi.

7.6 External (API Key)

Method	Endpoint	Auth	Deskripsi
GET	/api/v1/external/stats	API Key	Statistik agregat (total todo, selesai, kategori)

8. Format Response & Error Handling

8.1 Format Response Sukses

```
{
  "success": true,
  "message": "Todos fetched successfully.",
  "data": {
    "todos": []
  },
  "meta": {
    "total": 0,
    "page": 1,
    "limit": 10
  }
}
```

8.2 Format Response Error

```
{
  "success": false,
  "status": "fail",
  "message": "Todo not found.",
  "errors": []
}
```

8.3 Daftar HTTP Status Code

Kode	Arti
200	OK — request berhasil
201	Created — data berhasil dibuat
400	Bad Request — validasi input gagal
401	Unauthorized — token/API key tidak ada atau tidak valid
403	Forbidden — tidak memiliki izin/role yang cukup
404	Not Found — data tidak ditemukan
409	Conflict — data duplikat (contoh: email sudah terdaftar)
500	Internal Server Error — kesalahan tak terduga di server

9. Pengujian Manual dengan Postman

1. Buka aplikasi Postman.
2. Import file `postman/Todo-List-API.postman_collection.json` sebagai Collection.
3. Import file `postman/Todo-List-API.postman_environment.json` sebagai Environment.
4. Pilih environment "Todo List API - Local" di pojok kanan atas Postman.
5. Jalankan request secara berurutan di dalam tiap folder (Auth → Users → Categories → Todos → Activity Logs → External).
6. Token dan ID (token, adminToken, categoryId, todoid, userId) akan otomatis tersimpan ke environment variable melalui test script pada masing-masing request.

10. Dokumentasi API Interaktif (Swagger)

Setelah server dijalankan, dokumentasi interaktif dapat diakses melalui browser di alamat berikut:

```
http://localhost:5000/api-docs
```

Melalui halaman ini, seluruh endpoint dapat dicoba langsung (try it out), lengkap dengan skema request body, parameter, dan response yang diharapkan.

11. Troubleshooting (Masalah Umum)

Server gagal konek ke MongoDB

- Pastikan MongoDB sudah berjalan (mongod) atau MONGO_URI di .env mengarah ke instance yang benar (mis. MongoDB Atlas).

Error 401 Unauthorized

- Pastikan header Authorization terisi dengan format: Bearer <token>.
- Token mungkin sudah kedaluwarsa — login ulang untuk mendapatkan token baru.

Error 403 Forbidden

- Endpoint tersebut memerlukan role admin, sedangkan akun yang digunakan memiliki role user.

npm test gagal karena tidak bisa unduh binary MongoDB

- Pastikan koneksi internet tersedia, atau jalankan MongoDB lokal dan set MONGO_URI_TEST pada .env sebagai fallback.

12. Penutup

Dokumen ini disusun sebagai panduan teknis penggunaan Todo List Management API, mencakup instalasi, konfigurasi, menjalankan aplikasi, penggunaan endpoint, hingga pengujian. Untuk detail tambahan, silakan merujuk pada README.md di repository proyek atau dokumentasi Swagger yang tersedia.